

ESCAPE-Nav: Experience-Shaped Causally Aligned Perception–Execution for Asynchronous VLM Navigation

Anonymous Authors

Abstract—단일 전방 RGB 카메라 기반 navigation은 상대 PointGoal 또는 object goal이 주어지더라도 한 장의 영상만으로 주행 가능한 branch를 식별하기 어렵다. Vision–Language Model (VLM)은 사람처럼 필요한 방향을 살펴 local subgoal을 선택할 수 있지만, 느린 추론은 관측과 명령 적용 시점을 어긋나게 하며 실패 방향을 기억하지 않으면 deadlock 재진입과 oscillation이 반복된다. 본 논문은 이를 temporal, representational, experiential causal consistency 문제로 정식화하고 Experience-Shaped Causally Aligned Perception–Execution Navigation (ESCAPE-Nav)을 제안한다. ESCAPE-Nav는 task-conditioned directed/full sweep과 비동기 실행을 결합하고, VLM이 coarse task goal을 RGB에서 투영한 유한한 local candidate 중 하나로 refine하게 한다. 선택 pixel은 PixNav에, 동일 pixel의 robot-relative ground-plane point는 S2E에 전달한다. Delayed decision에는 goal representation에 따라 passive transform, viewpoint restoration 또는 재관측, apply-time metric rebase를 적용하고, identity, freshness, ownership 및 terminal 상태가 유효할 때만 원자적으로 handoff한다. Sparse directional graph는 acknowledgement된 transition의 failed entry와 reverse escape를 독립적으로 기록한다. 평가는 HM3D indoor와 Habitat-GS outdoor PointNav, NavBench-GS 재현, ObjectNav transfer, 세 VLM backbone 및 Unitree Go2 배치로 구성한다.

I. 서론

원격 탐사와 재난 대응 로봇은 정밀한 사전 지도 없이 coarse position 또는 semantic object goal을 향해 이동해야 한다. 전통적 system은 occupancy/cost map의 global path를 local subgoal로 분해하지만, 이 접근은 정확한 map–robot alignment에 의존한다. 반복 구조, 저특징 지형 및 Global Navigation Satellite System (GNSS)-denied 구간에서는 localization 오차가 global plan과 현재 관측을 어긋나게 한다. 이상적인 RGB-D+Global Positioning System (GPS)+Compass에서 높은 PointNav 성능도 realistic sensing과 actuation noise에 크게 저하된다 [1]–[3]. 본 연구는 occupancy, voxel, point cloud 또는 globally aligned semantic map을 유지하지 않고 monocular RGB, relative PointGoal 또는 object category, episode-local odometry만 사용하는 navigation을 다룬다.

이처럼 map alignment를 신뢰하기 어려운 조건에서는 현재 장면에서 주행 단서를 능동적으로 찾는 과정이 중요하다. 사람은 출발할 때 주변을 둘러보고, 교차로나 장애물 앞에서는 좌우와 목표 방향을 비교하며, 진행이 막히면 관찰 범위를 넓혀 다른 길을 택한다. 이는 필요한 시야를 확보하는 느린 판단과 이미 정한 방향으로 이동하는 빠른 실행을 병행하는 과정이다. 본 연구는 이 탐색 원리를 고정 전방 RGB 카메라, 느린 vision–language model (VLM)과 빠른 local navigation backend의 결합으로 구현한다. 그러나 이를 실제 주행 system으로 옮기려면 다음 세 난제를 함께 해결해야 한다.

첫 번째 난제는 *single-view partial observability*다. PointGoal은 거리와 방위를 주지만 그 방향이 현재 보이는 doorway

인지, 벽 뒤의 막다른 공간인지, 우회가 필요한 branch인지는 알려주지 않는다. 고정 전방 카메라가 정면만 반복 관측하면 측면 출구와 목표 주변의 자유 공간도 놓친다. 이러한 능동적 시야 선택은 active perception의 원리와 일치하지만 [4], [5], VLM navigation에서 view acquisition과 실제 주행을 서로 다른 시간 척도로 결합하는 문제는 충분히 다루이지 않았다 [6].

두 번째 난제는 VLM의 *delayed semantic decision*이다. VLM은 traversable floor나 semantic branch를 고를 수 있지만 local policy보다 느리다. 동기 실행은 추론마다 stop-and-go motion을 만들고, 단순 비동기 실행은 관측 view와 적용 시점의 robot state를 불일치시킨다. 또한 world point, robot-relative metric subgoal 및 image-bound pixel은 서로 다른 물리적 의미를 가지므로 오래된 pixel을 현재 영상에 복사하거나 이미 수행한 회전을 반복해서는 안 된다.

세 번째 난제는 *failure provenance*의 보존이다. 막다른 방 전체를 위험하게 표시하면 성공한 탈출 방향까지 잃지만, 최근 image/action만 prompt에 추가해서는 “복도에서 방으로 들어간 방향은 실패했고 반대 방향 탈출은 성공했다”는 비대칭 관계를 안정적으로 보존하기 어렵다. 그 결과 같은 branch의 deadlock 재진입과 저진행 heading reversal 또는 $A \rightarrow B \rightarrow A$ 형태의 oscillation이 반복된다.

본 논문은 이를 위해 필요한 view를 능동적으로 획득하고 perception–execution chain의 인과 정합성을 보존하는 Experience-Shaped Causally Aligned Perception–Execution Navigation (ESCAPE-Nav)을 제안한다. 여기서 *experience-shaped*는 확인된 directional outcome이 이후 branch 선택을 바꾼다는 뜻이다. **Temporal consistency**는 decision을 원래 observation과 실제 적용 시점에 연결한다. **Representational consistency**는 pixel · metric · world goal에 각 의미에 맞는 변환을 적용하고, **experiential consistency**는 확인된 outcome을 실제 실행한 directed action에 귀속한다. ESCAPE-Nav는 저주기 VLM supervisor, active-observation scheduler, 고주기 backend와 sparse directional memory로 구성된다. ObjectNav는 초기 full sweep, PointNav는 goal-aligned directed sweep을 수행하고, candidate가 없거나 no-progress/oscillation이 감지되면 시야를 확장한다. 추론 중에도 backend는 마지막 승인 subgoal을 계속 추종한다.

Coarse GPS final goal은 state estimator를 통해 relative PointGoal로 제공된다. VLM은 그 거리 · 방향, multi-view RGB와 memory를 이용해 현재 보이는 executable candidate 중 하나를 fine local goal로 선택한다. 각 decision은 observation bundle, capture pose/time, episode/goal identity 및 expected owner에 결합된다. World goal은 현재 body frame으로 passive transform하고, capture view의 pixel은 viewpoint를 복원하거나 재관측한다. 동일 pixel의 ground-plane point는 S2E context

frame으로 rebase한 뒤 실제 application pose에서 trajectory를 정렬한다. Acknowledgement된 $A \rightarrow B$ 실패와 $B \rightarrow A$ 탈출은 독립적인 directed edge로 기록하여 실패 진입만 억제한다. Fig. 1가 전체 구조를 요약한다.

본 논문의 기여는 다음과 같다.

- **Active observation과 인과 결합 비동기 실행.** Task-conditioned directed/full sweep을 inference-execution overlap 및 identity, freshness, ownership, terminal fence와 결합해 delayed result를 fail-closed한다.
- **Representation-aware transport.** World point, robot-relative metric goal 및 view-bound pixel을 구분하고 각각 passive transform, apply-time rebase, viewpoint restoration 또는 재관측을 적용한다.
- **방향성 memory와 재학습 없는 통합.** Acknowledged failed entry와 reverse escape를 독립적으로 저장해 반복 실패를 억제하고, 공통 action/outcome contract로 PixNav와 reinforcement learning (RL) post-training한 S2E를 추가 joint tuning 없이 연결한다.

평가는 HM3D indoor와 Habitat-GS outdoor의 paired PointNav를 중심으로 active observation, causal transport, directional memory 및 VLM backbone을 분해한다. NavBench-GS는 별도 S2E 재현, ObjectNav는 goal-modality transfer, Unitree Go2는 deployment validation에 사용하며 서로 다른 task의 published metric을 직접 혼합하지 않는다.

II. 관련 연구

A. PointNav와 localization robustness

Habitat와 HM3D는 photorealistic PointNav의 대표 평가 기반이다 [7]–[9]. DD-PPO의 이상적 RGB-D+GPS+Compass 성능과 달리 realistic sensing/actuation noise는 성능을 크게 낮추며 [1], Visual Odometry (VO)와 augmentation을 결합한 연구가 더 현실적인 baseline을 제공한다 [2], [3]. 본 연구의 직접 비교에는 depth를 제외한 DD-PPO RGB와 공개 RGB 기반 VO agent를 사용한다.

B. Foundation-model navigation과 active perception

PixNav는 foundation model이 선택한 pixel을 RGB local skill에 연결하고 [10], S2E는 web-video pretraining, simulation RL 및 outdoor 3D Gaussian Splatting benchmark NavBench-GS를 제안했다 [11]. Habitat-GS는 별도의 Habitat-compatible outdoor 평가 환경이다 [12]. Active perception은 task에 맞게 sensor pose를 선택하며 [4], [5], ImagineNav는 미래 view를 이용한 VLM ObjectNav를 탐구했다 [6]. ESCAPE-Nav는 새 view generator를 학습하지 않고 fixed forward camera의 current/directed/full sweep을 선택하고 각 view를 capture pose와 identity에 결합한다.

C. Slow-fast asynchronous navigation

AsyncVLA는 vision-language-action (VLA) model과 on-board adapter를 비동기로 실행하고 [13], SFCo-Nav는 vision-language navigation (VLN)의 slow reasoning과 fast graph alignment를 결합한다 [14]. AsyncShield는 pose buffer와 kinematic mapping으로 stale metric subgoal을 변환한다 [15]. ESCAPE-Nav는 pose rebase 자체보다 multi-view identity, representation별 transport, command ownership과 terminal lifecycle을 포함한 fail-closed transaction을 directional memory와 결합한다.

D. Navigation memory와 recovery

ReMemNav는 episodic memory로 ObjectNav deadlock을 완화하고 [16], EvoMemNav는 visual-semantic graph와 reflection-driven write-back을 사용한다 [17]. ESCAPE-Nav는 실제 handoff된 transition과 terminal outcome만 저장하여 failed entry와 reverse escape를 분리하고, 불확실한 place association은 hard merge 대신 duplicate node와 soft link로 보존한다.

III. 방법

A. 문제 정의와 공통 구성

시간 t 의 robot body frame B 에 대한 episode-local planar pose를 $\mathbf{T}_{WB}(t) \in \text{SE}(2)$ 라 하자. W 는 episode 시작점의 local odometry frame이며, t 는 timestamp, 굵은 $\mathbf{T}$ 는 pose/transform에만 사용한다. k 번째 decision의 observation, S2E RGB context의 마지막 frame 및 application anchor는

$$\begin{aligned} t_k^O, t_k^C, t_k^A, \\ \mathbf{T}_k^O := \mathbf{T}_{WB}(t_k^O), \quad \mathbf{T}_k^C := \mathbf{T}_{WB}(t_k^C), \quad \mathbf{T}_k^A := \mathbf{T}_{WB}(t_k^A) \end{aligned} \quad (1)$$

로 구분한다. 입력은 전방 monocular RGB I_t , PointNav relative goal $\tilde{\mathbf{g}}_t = (\tilde{\rho}_t, \tilde{\phi}_t)$ 또는 ObjectNav category c_g , local odometry이며, $\tilde{\rho}_t$ 와 $\tilde{\phi}_t$ 는 body-frame goal distance와 bearing이다. Persistent dense map은 사용하지 않고 sparse experience graph만 유지한다.

ESCAPE-Nav는 저주기 VLM supervisor Φ , 중간 주기 active-observation scheduler, 고주기 executor π_E 와 controller로 구성된다. k 번째 query는

$$\mathbf{q}_k = (O_k, g_k, \mathcal{M}_k, \mathbf{T}_k^O, t_k^O, \mathbf{u}_k), \quad (2)$$

이다. $O_k = \{(I_{k,j}, \delta\psi_{k,j}, t_{k,j})\}$ 는 image와 relative heading, g_k 는 canonical task goal, $\mathcal{M}_k$ 는 memory snapshot이다. $\mathbf{u}_k$ 는 episode, task, goal version과 observation sequence를 묶는 immutable identity다. VLM은

$$\mathbf{d}_k = (\alpha_k, r_k, \mathbf{z}_k, c_k, t_k^D, \mathbf{u}_k), \quad (3)$$

$$\alpha_k \in \{\text{go}, \text{rotate}, \text{request_observation}, \text{stop}\}, \quad (4)$$

를 출력한다. r_k 는 representation, $\mathbf{z}_k$ 는 candidate reference와 selected pixel, c_k 는 confidence/validity, t_k^D 는 completion time이다. Metric point는 backend가 calibration으로 계산하며 VLM은 자유 좌표나 low-level command를 생성하지 않는다. 이 유한 후보 선택은 hallucinated coordinate를 막고, 하나의 semantic choice를 서로 다른 backend 표현으로 재현 가능하게 전달한다.

a) *Prompted semantic supervision and structured contract.*: Canonical input은 task goal/state, capture metadata가 결합된 ordered RGB views, candidate set, memory snapshot과 action constraint다. Strict output은 Eq. (4)의 action, grounded view/candidate/pixel, confidence, time-to-live (TTL), 선택적 observation/memory proposal을 반환한다. Prompt는 PointNav의 geometric stop과 ObjectNav의 visual confirmation을 구분하고 미제공 candidate/coordinate 생성을 금지한다. Backend는 identity, freshness, ownership, terminal state와 geometry를 재검증하며 memory는 acknowledged outcome만 commit한다.

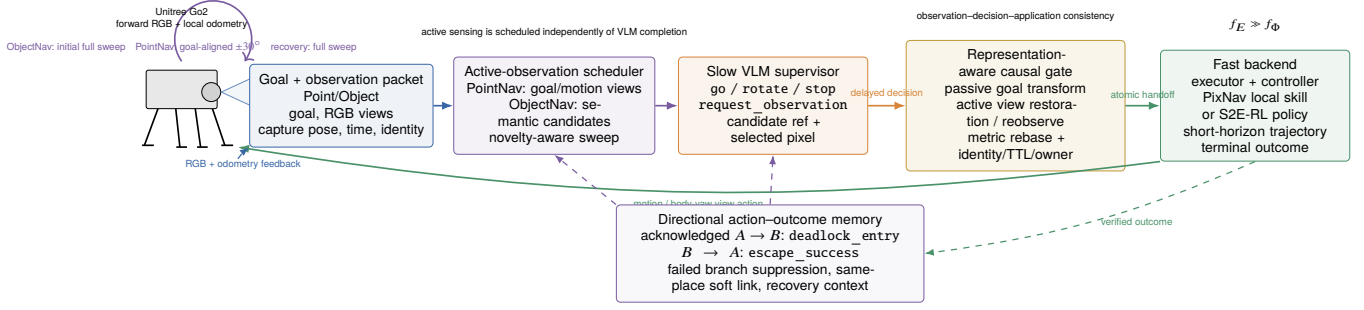


Fig. 1. ESCAPE-Nav의 system overview. 고정 전방 카메라를 가진 Go2는 task-conditioned active view를 수집하고, VLM 추론 중에도 마지막으로 승인된 subgoal을 추종한다. Delayed output은 goal representation에 따라 passive transform, active viewpoint restoration 또는 metric rebase를 거쳐야 하며, 실제 terminal acknowledgement만 directional memory를 갱신한다.

TABLE I
관련 계열의 중심 MECHANISM 비교. “-”는 해당 논문의 주된 CLAIM이 아님을 뜻한다.

방법	주 task	Active view	비동기 slow-fast	apply-time transform	view-bound goal 처리	방향성 failure memory	통합 joint tuning
DD-PPO [1]	PointNav	-	-	-	-	-	정책 학습
PixNav [10]	Local/ObjectNav	VLM pixel	-	-	capture pixel	-	skill 사전학습
ImagineNav [6]	ObjectNav	imagined view	-	-	image selection	-	module 학습
AsyncVLA [13]	실로봇 goal/language nav	제한적	사용	learned adapter	learned alignment	-	필요
AsyncShield [15]	cloud VLA nav	-	사용	kinematic mapping	metric subgoal	-	edge adapter RL
SFCo-Nav [14]	VLN	trigger 기반	사용	-	graph alignment	attributed graph	불필요
ReMemNav/EvoMemNav [16], [17]	Object/GOAT Nav	선택적	-	-	semantic view	semantic/topological	불필요
ESCAPE-Nav	PointNav 중심	task-conditioned sweep	사용	passive+active+rebase	restore/reobserve	action-outcome edge	불필요

State estimator는 GPS final goal을 relative PointGoal ($\tilde{p}_t, \tilde{\phi}_t$)로 변환하지만, 이는 따라야 할 doorway나 floor branch를 정하지 않는다. Candidate generator는 lower-image grid를 deterministic하게 샘플링하고 calibrated ray-ground-plane intersection으로 pixel을 capture-time metric point에 결합한 뒤 투영/RGB screening 실패를 제거한다. Φ 는 PointGoal, multi-view evidence와 memory로 coarse final goal을 한 local candidate로 refine한다. 동일 candidate를 PixNav에는 pixel, S2E에는 metric point로 전달한다. Grid와 cap은 Sec. IV-C에 공개하며, 선택점은 collision-free certificate가 아니라 locally executable hypothesis다.

B. 공통 active observation과 goal binding

Observation mode는 current, directed_view, directed_sweep, full_sweep이며, 그 집합과 scheduler 선택을 각각 $\mathcal{U}_O$ 와 $\mu_t \in \mathcal{U}_O$ 로 쓴다. Validated request를 $o_t^{\text{req}} \in \mathcal{U}_O \cup \{\emptyset\}$ 라 하면,

$$\mu_t = \begin{cases} \text{full_sweep}, & t = 0 \wedge \text{ObjectNav}, \\ \text{directed_sweep}, & t = 0 \wedge \text{PointNav}, \\ o_t^{\text{req}}, & o_t^{\text{req}} \in \mathcal{U}_O, \\ \text{current}, & \text{otherwise}, \end{cases} \quad (5)$$

로 선택한다. ObjectNav는 목표가 어느 방향에나 있을 수 있어 초기 full sweep을 사용하고, PointNav는 알려진 goal bearing 주변만 확인해 관측 비용을 줄인다. Candidate 부재, blocked/no-progress, oscillation 또는 place ambiguity가 생길 때만 시야를 확장하며, 초기 PointNav candidate가 비면 한 번 full sweep한다. World-yaw novelty ledger는 같은 pose의 반복 관측을 막고, yaw bin이 소진되면 translation recovery 또는 다른 branch로 전환한다. Sweep 값은 Sec. IV-C에 명시한다.

Sync와 Async는 observation, novelty 및 grounding을 공유한다. Body-yaw capture가 잠시 motion을 점유한 뒤 Sync는 새 decision까지 정지하고, Async는 마지막 trajectory를 재개해 inference와 중첩한다. 각 bundle은 자체 timestamp와 capture pose를 갖는다.

두 실행 방식은 다음 representation contract를 공유하고, Async만 pose anchor 사이 transport를 추가한다.

World-anchored goal. 고정점 ${}^W\mathbf{g}$ 는 현재 body frame으로

$$\begin{bmatrix} B_t \mathbf{g} \\ 1 \end{bmatrix} = \mathbf{T}_{WB}(t)^{-1} \begin{bmatrix} {}^W\mathbf{g} \\ 1 \end{bmatrix} \quad (6)$$

의 passive transform으로 갱신하며 Habitat PointGoal도 같은 robot-relative 의미를 갖는다.

View-anchored pixel. $\mathbf{u}_{k,j}$ 는 image, capture heading과 identity에 결합되므로 PixNav는 선택 view를 복원하거나 재 관측하며 다른 view에 좌표를 복사하지 않는다.

Robot-relative metric goal. Candidate pixel은 ray-ground-plane intersection으로 ${}^{Bo}\mathbf{z}_k$ 에 결합되고 S2E가 short-horizon trajectory를 생성한다. 이는 view-bound pixel과 transport가 능한 metric point를 분리한다.

C. Synchronous reference system

Fig. 2(a)의 Sync는 O_k 획득 후 $\mathbf{d}_k$ 가 반환될 때까지 navigation을 보류한다. Contract 검증 뒤 PixNav에는 capture view/pixel, S2E에는 동일 pixel의 metric point를 전달하고, acknowledged outcome만 memory에 기록한다. Concurrent translation과 non-blocking handoff가 없어 흐름은 명확하지만 VLM latency만큼 wall time이 늘고 duty cycle이 낮아진다. 따라서 Sync는 최종 방식이 아니라 공통 dataflow를 고정하고 Async에서 달라지는 시간 정렬만 분리해 설명하기 위한 reference다.

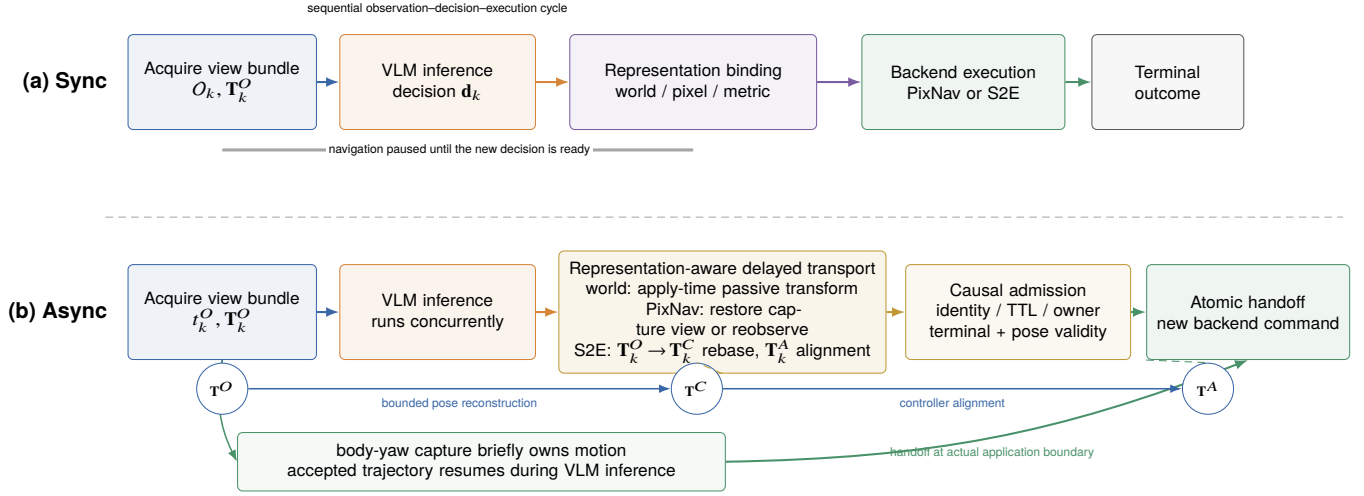


Fig. 2. Synchronous reference와 causally aligned asynchronous pipeline의 비교. Body-yaw capture 동안에는 observation primitive가 motion을 점유한다. 이후 Sync는 새 decision까지 정지하지만, Async는 기존 trajectory를 재개해 VLM inference와 중첩한다. Delayed output은 pose reconstruction, representation-aware transport와 causal admission을 거쳐 실제 application boundary에서 원자적으로 handoff된다.

D. Causally aligned asynchronous extension

Fig. 2(b)의 Async는 Sync의 stage를 유지하면서 capture 후 기존 trajectory와 inference를 중첩하고, Eq. (1)의 세 anchor를 이용해 delayed decision을 실제 application boundary에서 교체한다.

1) *Concurrent execution and timestamped pose reconstruction*: Body-yaw capture interval의 집합을 $\mathcal{T}_O$ 라 하자. Observation control policy π_O 와 fast executor π_E 사이의 motion ownership은

$$\mathbf{u}_t = \begin{cases} \pi_O(\mu_t), & t \in \mathcal{T}_O, \\ \pi_E(I_{t-h:t}, \bar{\mathbf{z}}_k, \hat{\mathbf{x}}_{t-h:t}), & t \notin \mathcal{T}_O, \end{cases} \quad f_E \gg f_\Phi. \quad (7)$$

여기서 $\mathbf{u}_t$ 는 commanded control, π_O 는 bounded body-yaw policy, h 는 history horizon, k^- 와 $\bar{\mathbf{z}}_k$ 는 마지막 admission index와 transported subgoal, $\hat{\mathbf{x}}_{t-h:t}$ 는 estimated state history다. f_E 와 f_Φ 는 executor와 supervisor의 frequency이며 $f_E \gg f_\Phi$ 다. Capture 후 π_E 가 기존 subgoal을 재개한다. 비동기 query의 관측 · context · 적용 시각은 odometry sample과 일반적으로 일치하지 않으므로, 단순 latest-pose 조치는 이동 중 공간 오차를 만든다. 이에 $t^- < t < t^+$ 인 같은 frame의 $\mathbf{p}^- = (x^-, y^-, \theta^-)$ 와 $\mathbf{p}^+ = (x^+, y^+, \theta^+)$ 를 $\lambda = (t - t^-)/(t^+ - t^-)$ 로 보간한다.

$$\hat{\mathbf{p}}(t) = \begin{bmatrix} (1 - \lambda)x^- + \lambda x^+ \\ (1 - \lambda)y^- + \lambda y^+ \\ \mathcal{W}_\pi(\theta^- + \lambda \mathcal{W}_\pi(\theta^+ - \theta^-)) \end{bmatrix} \quad (8)$$

로 translation은 선형, yaw는 shortest angle로 보간한다. $\mathcal{W}_\pi: \mathbb{R} \rightarrow (-\pi, \pi]$ 는 angle wrapping이다. Bracket 부재, frame 불일치 또는 과도한 간격에는 외삽하지 않으며 bound는 Sec. IV-C에 둔다. 이는 image/trajectory가 아니라 timestamped pose의 synchronization이다.

2) *Representation-aware delayed transport*: World goal은 Eq. (6)을 application pose에서 다시 계산한다. Pixel은 passive rebase할 수 없으므로 capture yaw $\psi_{k,j}^O$ 를

$$\delta\psi_k^{\text{view}} = \mathcal{W}_\pi(\psi_{k,j}^O - \psi_k^A) \quad (9)$$

로 복원해 quantize한다. Translation/scene change가 bound를 넘으면 pixel을 폐기하고 재관측한다.

S2E metric goal은 observation frame에서 RGB context frame으로 한 번 rebase한다. B_O, B_C, B_A 는 각 anchor의 body frame이다.

$$\begin{bmatrix} B_C \mathbf{z}_k \\ 1 \end{bmatrix} = (\mathbf{T}_k^C)^{-1} \mathbf{T}_k^O \begin{bmatrix} B_O \mathbf{z}_k \\ 1 \end{bmatrix}. \quad (10)$$

Trajectory frame은 $\mathbf{T}_k^C$ 로 고정하고 application state를

$$\mathbf{T}_{B_C B_A, k} = (\mathbf{T}_k^C)^{-1} \mathbf{T}_k^A \quad (11)$$

로 표현한다. Controller가 velocity와 acknowledged progress를 반영해 실제 handoff에서 연결한다. PixNav의 $\hat{\mathbf{T}}_k^{\text{sync}}$ 는 관측 시점의 승인 경로를 동일한 elapsed action budget만큼 실행했다면 도달했을 counterfactual Sync pose다. 실제 $\mathbf{T}_k^A$ 에서는 그 pose까지의 bounded residual만 실행하며, 이 예측값은 Eq. (8)의 측정 odometry pose를 대체하지 않는다.

3) *Admission and non-blocking handoff*: Decision이 application time t_k^A 에 도착하면

$$\Gamma(\mathbf{d}_k, t_k^A) = \mathbb{I}[t_k = \mathbf{t}(t_k^A)] \mathbb{I}[t_k^A - t_k^O \leq \tau_{\max}] \cdot \mathbb{I}[\omega_k^{\text{exp}} = \omega(t_k^A)] \mathbb{I}[b_T(t_k^A) = 0] \mathbb{I}[b_{P, k} = 1] \quad (12)$$

$\Gamma = 1$ 만 admission한다. $\mathbb{I}[P]$ 는 indicator, $\mathbf{t}(t_k^A)$ 는 active identity, $\tau_{\max}$ 는 TTL, ω_k^{exp} 와 $\omega(t_k^A)$ 는 expected/current owner다. b_T 는 terminal, $b_{P, k}$ 는 pose와 displacement validity flag다. Identity/goal 변경, expiration, owner mismatch, terminal 또는 invalid pose는 fail-closed하며, 이 gate는 physical safety가 아닌 causal admissibility를 보장한다.

rotate 명령은 latency 동안 이미 수행된 yaw를 제외한 residual만 실행한다.

$$\Delta\bar{\psi}_k = \mathcal{W}_\pi(\Delta\psi_k - (\psi_k^A - \psi_k^O)). \quad (13)$$

$\Delta\psi_k$ 는 requested yaw, 괄호 안 두 번째 항은 이미 수행한 yaw, $\Delta\bar{\psi}_k$ 는 residual이다. Admission 후 atomic handoff와

owner-matched acknowledgement를 사용한다. 따라서 Eq. (8)의 reconstruction, Eqs. (9)–(11)의 transport와 Eq. (12)가 causal binding을 보존한다.

E. Directional action–outcome memory와 recovery

Memory는 sparse directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 다. Node v_i 는 keyframe, semantic description, pose belief, visit count와 recovery state를 보존한다. 구현은 RGB의 deterministic hash-projection descriptor f_{pr} 와 cosine similarity로 revisit 후보만 검색하고, semantic 비교와 relative-pose plausibility가 모두 맞을 때 same-place를 승인한다. 불확실하면 duplicate node와 soft link를 유지한다.

Directed edge는

$$e_{ij} = (v_i, v_j, \Delta \hat{\mathbf{T}}_{ij}, y_{ij}, n_{ij}^+, n_{ij}^-, \pi_{ij}) \quad (14)$$

이다. $\Delta \hat{\mathbf{T}}_{ij}$ 는 relative transform, y_{ij} 는 success, blocked, deadlock_entry, escape_success 등의 outcome, n_{ij}^+, n_{ij}^- 는 acknowledged count다. Provenance π_{ij} 는 decision/command ID, pose, time, owner와 terminal evidence를 담는다. e_{ij} 와 e_{ji} 는 독립이며 graph는 handoff 후 acknowledged outcome만 갱신한다.

최근 execution evidence에서 blocked flag b_t , collision count N_t^{col} , translation-stall count N_t^{stall} , heading-oscillation hit N_t^{osc} 및 graph-cycle hit N_t^{cyc} 을 계산한다.

$$\begin{aligned} D_t = \mathbb{I} [& b_t = 1 \vee N_t^{\text{col}} > 0 \\ & \vee N_t^{\text{stall}} \geq K_s \\ & \vee \{N_t^{\text{stall}} \geq K_o \wedge N_t^{\text{osc}} > 0\} \\ & \vee N_t^{\text{cyc}} > 0]. \end{aligned} \quad (15)$$

D_t 는 recovery를 여는 deadlock trigger다. N_t^{stall} 은 이동량 $\leq \epsilon_s$ 인 연속 no-progress count, K_s, K_o 는 stagnation/oscillation threshold다. N_t^{cyc} 는 이동 중 재방문도 검출하므로 Eq. (15)은 hard failure와 bounded recurrence를 함께 사용한다. 값은 Sec. IV-C에 공개하며 online trigger와 stored-edge penalty는 구분한다.

Acknowledged negative outcome set $\mathcal{Y}^-$ 는 deadlock_entry, deadlock_entry_candidate, blocked, risky를 포함한다. Candidate c 가 참조하는 edge의 outcome과 마지막 reason을 각각 y_c 와 ℓ_c 라 하면

$$\begin{aligned} A_{\text{goal}}(c) &= 1 - \frac{|\mathcal{W}_{180}(\beta_c - \tilde{\phi}_t)|}{180}, \\ C_{\text{dead}}(c) &= \mathbb{I}[y_c \in \mathcal{Y}^-], \\ C_{\text{osc}}(c) &= \mathbb{I}[\ell_c = \text{loop}] \min(c_{\text{max}}, \alpha_o n_c^-), \\ S_{\text{mem}}(c) &= \lambda_a A_{\text{goal}}(c) - \lambda_d C_{\text{dead}}(c) \\ &\quad - \{1 - C_{\text{dead}}(c)\} C_{\text{osc}}(c), \\ S_{\text{front}}(c) &= \lambda_a A_{\text{goal}}(c) + \lambda_f. \end{aligned} \quad (16)$$

β_c 는 bearing, $\mathcal{W}_{180}$ 은 degree를 $(-180, 180]$ 로 감싸는 연산, n_c^- 는 failure count, $\ell_c = \text{loop}$ 는 topological_loop_candidate다. c_{max}, α_o 는 loop penalty의 cap/rate이고 $\lambda_a, \lambda_d, \lambda_f$ 는 alignment, negative-edge penalty, frontier prior다. Negative edge에는 λ_d , 나머지에만 C_{osc} 를 적용해 중복을 피한다. S_{front} 는 unvisited view에만 사용하고 값은 Sec. IV-C에 둔다. 실행 가능한 branch가

없으면 observation/recovery를 요청하며, 탈출 후 reverse edge에 escape_success를 기록한다.

Fig. 3처럼 실패는 destination이 아닌 acknowledged directed edge에만 귀속되어 reverse escape와 다른 branch를 보존한다.

F. Backend contract와 구현 세부사항

Backend는 PixNav local skill 또는 S2E-RL policy이고, executor는 backend 호출, ownership, handoff와 terminal acknowledgement를 관리한다. Contract는 Eq. (4)의 action, representation metadata와 verified outcome으로 구성된다. 두 adapter는 같은 candidate identity를 공유하며 PixNav는 capture view/pixel, S2E는 결합된 metric point를 소비한다. 이하 ESCAPE-Nav에 PixNav와 S2E-RL을 연결한 instantiation을 각각 ESCAPE-PixNav와 ESCAPE-S2E로 표기한다. 별도 variant 표시가 없으면 active observation, causal transport와 directional memory를 모두 사용하는 full system이다. PixNav Native는 독립 baseline이고 ESCAPE-PixNav는 동일 skill에 ESCAPE-Nav supervisor, scheduler와 memory를 결합한다.

S2E-RL은 공개 S2E-BC를 초기화로 저자가 RL post-training한 backend다. Residual-Attention Module (RAM), proximal policy optimization (PPO), reward/curriculum의 재현 범위는 실험에 명시한다. ESCAPE-Nav결합 시 추가 gradient update가 없으므로 training-free는 integration에만 한정한다.

Unitree Go2 runtime은 고정 전방 RGB, Robot Operating System (ROS) [18] odometry와 body-yaw primitive를 사용한다. Contract의 right-turn-positive degree를 adapter 경계에서 ROS/controller의 counter-clockwise-positive radian으로 한 번 변환한다. Simulator pose를 clean oracle로 쓸 때는 명시하고 모든 robustness method에 같은 corruption trace를 적용한다.

IV. 실험 설계

평가는 다음 research questions (RQs)를 검증한다. **RQ1:** goal-conditioned active observation이 single-view baseline 보다 branch selection과 recovery를 개선하는가? **RQ2:** representation-aware transport와 causal admission이 VLM latency에서 stale application과 stop-and-go cost를 줄이는가? **RQ3:** directional memory가 failed-branch re-entry와 oscillation을 줄이는가? **RQ4:** 동일 contract가 PixNav와 독립적으로 RL post-training한 S2E에 joint tuning 없이 적용되며, 두 backend의 indoor/outdoor domain 특성이 어떻게 다른가? **RQ5:** 동일 supervisor contract가 model-specific prompt tuning 없이 서로 다른 VLM backbone에서 유효한 grounded decision을 생성하는가? **RQ6:** 전체 system이 Unitree Go2에서 실시간으로 동작하는가?

Primary evidence는 RQ1–RQ3의 세 mechanism과, 각 environment에서 동일 backend의 Direct-goal variant와 ESCAPE-Nav를 짝지은 RQ4 비교다. VLM portability와 ObjectNav transfer는 generality를, NavBench-GS는 S2E evaluator calibration을 보조하며 primary claim의 실패를 대체하지 않는다. Go2 결과는 실제 runtime에서 causal event sequence가 성립하는지를 검증하되 적은 trial로 환경 generalization을 주장하지 않는다.

A. 환경과 평가 subset

HM3D PointNav. HM3D validation split에서 method 실행 전에 고정된 $N = 200$ episode를 사용한다 [9].

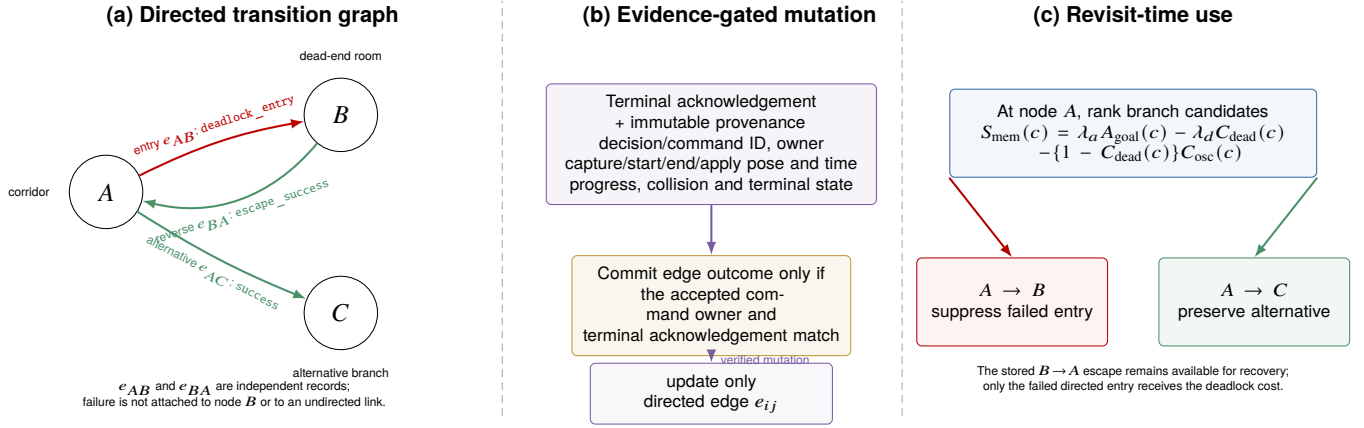


Fig. 3. Directional action–outcome memory. (a) Entry failure, reverse escape and an alternative branch are stored as independent directed edges. (b) A graph mutation requires terminal acknowledgement whose owner matches the accepted command, and the edge retains the associated temporal and spatial provenance. (c) On revisit, the failed entry is penalized without blacklisting the destination node, the reverse escape, or other executable branches.

Episode는 최소 20개 scene에 분산하고 shortest-path length와 geodesic/Euclidean ratio를 기준으로 층화한다. 모든 방법은 동일 episode identifier, start pose, goal, action budget, VLM decoding seed 및 noise trace를 사용한다. 결과를 본 뒤 episode를 제외하지 않으며 simulator failure만 사전 정의한 rule로 모든 방법에서 함께 제외한다. 계산 제약으로 최종 N 이 100인 경우에는 leaderboard-scale 평가가 아닌 “fixed 100-episode validation subset”으로 명시하고 큰 effect만 주장한다.

Habitat-GS PointNav. Outdoor 주 평가는 Habitat-GS의 고정 scene/episode subset에서 수행한다 [12]. HM3D와 동일한 relative PointGoal interface, success threshold, RGB-only observation, action budget 및 corruption을 사용하고 DD-PPO RGB, Strong VO, Direct-goal PixNav/S2E-RL, ESCAPE-PixNav 및 ESCAPE-S2E를 비교한다. Outdoor adaptation이 없는 frozen indoor DD-PPO/VO는 참고 결과이며, primary comparison은 같은 frozen backend의 Direct-goal 대 ESCAPE-Nav다. Interface failure는 setup 오류로 수정한 뒤 전체 방법을 재실행한다. Scene/episode와 start/goal manifest를 고정 공개하며 NavBench-GS 결과와 합산하지 않는다.

Controlled recovery. Random episode에서 deadlock 발생 수가 적을 수 있으므로 40개의 사전 정의 recovery episode를 별도로 사용한다. Goal-aligned dead end, blocked direct branch, symmetric corridor, visually aliased junction, latency spike 및 dynamic crossing을 여러 pose/seed에서 반복한다. 각 episode에는 deadlock 구간, 유효 탈출 branch와 revisit opportunity를 사전에 annotation하여 detector miss와 false trigger를 recovery 성공과 분리한다. Repeated-corridor 및 return-to-junction episode에는 same-place relation도 annotation하여 place retrieval과 최종 승인 정확도를 평가한다.

Candidate-grounding audit. Coarse PointGoal이 실제 실행 가능한 local goal로 refine되는지를 end-to-end SPL만으로 간접 주장하지 않는다. HM3D와 Habitat-GS의 paired PointNav episode에서 Direct-goal, ESCAPE-PixNav (1V) 및 ESCAPE-PixNav가 선택한 candidate를 simulator의 hidden navigation mesh로 사후 평가하며, 이 hidden state는 method 입력에 사용하지 않는다. 선택 candidate가 navigable surface에 있고 backend horizon의 collision-free rollout이 가능한 비율

을 candidate validity로, rollout 뒤 geodesic goal distance가 감소한 비율을 progress rate로 정의한다. 같은 query의 executable candidate 중 최선과 선택 candidate의 residual geodesic distance 차이는 selection regret으로 보고한다. 세 지표는 candidate가 존재했던 query와 empty-candidate query를 분리하고 environment/backend별 paired confidence interval을 제시한다.

ObjectNav transfer. HM3D ObjectNav의 고정 100 episode에서 target category를 VLM에 제공하고 PixNav Native-6V, PixNav Native-1V, ESCAPE-PixNav (1V), ESCAPE-PixNav 및 ESCAPE-S2E를 평가한다 [19]. 이는 PointNav와 동등한 main leaderboard가 아니라 active observation, memory 및 backend contract의 goal-modality transfer를 검증한다.

NavBench-GS. S2E 재현 평가는 원 논문의 NavBench-GS와 동일한 26개 scene, start/goal, robot embodiment, controller, collision geometry, timeout, success definition 및 네 condition (Empty, Obstacle, Pedestrian, Obstacle+Pedestrian)을 사용한다 [11]. 이 결과는 Habitat-GS outdoor 주 평가와 분리하여 Table IV에 독립적으로 보고한다. 보조 결과임을 반영하여 네 condition의 equal-weight macro-average만 제시하고 published row와 our evaluation row를 구분한다.

Real Unitree Go2. 실로봇은 main 통계 benchmark가 아니라 deployment validation이다. Dead-end room, blocked goal direction과 repeated corridor의 세 core scenario에서는 동일 route/start/goal에서 사전 고정된 same-backend Direct-goal reference와 Full ESCAPE-Nav를 각각 5회 paired 반복한다. Active-view recovery와 dynamic obstacle은 Full system을 각각 5회 실행해 배치 범위를 확인하되 baseline 우월성이나 generalization의 근거로 사용하지 않는다. 성공, intervention-free completion, recovery count, failed-edge re-entry, VLM latency와 motion duty cycle을 기록하고, paired route의 RGB, trajectory와 event log를 video로 제공한다.

B. 비교 방법

HM3D PointNav baselines.

- 1) **DD-PPO RGB:** RGB+GPS+Compass PointNav agent를 외부 learned-policy reference로 사용한다 [1]. Depth를 사용하는 DD-PPO result는 본 비교에서 제외한다.

2) *DD-PPO RGB under corruption* 및 *Strong VO PointNav*: test-time localization sensitivity와 noise-aware realistic policy를 비교한다 [2], [3]. Strong VO는 최종 평가 전에 단일 공개 method/checkpoint와 sensor input을 고정하며, 재현 가능한 checkpoint를 확보하지 못하면 일반명 baseline을 유지하지 않는다.

3) *Direct-goal PixNav*: current view의 executable candidate 중 PointGoal bearing error가 가장 작은 pixel을 선택하며, 후보가 없으면 goal 방향으로 deterministic rotate한다. *Direct-goal S2E-RL*: 현재 robot-relative PointGoal을 backend horizon에 맞게 제한해 metric goal로 전달한다. 두 방법 모두 VLM, active sweep 및 memory를 사용하지 않으며 tie-break와 horizon은 run manifest에 고정한다.

4) *ESCAPE-PixNav (1V)*: ESCAPE-PixNav와 동일한 supervisor contract, directional memory, async transport/admission 및 frozen PixNav backend를 유지하되, active acquisition만 제거하고 각 query의 최신 전방 RGB 한 장을 사용한다. Localization-degraded track에서도 다른 방법과 동일한 corrupted relative goal, odometry 및 RGB stress trace를 입력받는다. 이 variant는 candidate-grounding audit, ObjectNav transfer 및 active-observation ablation에서 사용하고, PointNav 주 결과는 동일 backend의 Direct-goal 대 full ESCAPE-Nav비교에 집중한다.

5) *ESCAPE-PixNav*와 *ESCAPE-S2E*: 동일 supervisor contract와 mechanism을 서로 다른 frozen backend에 적용한다. PixNav Native-6V/1V는 ObjectNav baseline이므로 이 비교와 Table III에서 제외한다.

HM3D ObjectNav baselines. *PixNav Native-6V*는 원 protocol처럼 단일 전방 RGB camera를 제자리 회전하며 순차 취득한 여섯 view의 prompting과 pixel skill을 유지한다 [10]. *PixNav Native-1V*는 PixNav의 prompt, decision interface와 skill을 유지하되 입력만 최신 전방 view 한 장으로 제한하여 native sweep의 효과를 분리한다. *ESCAPE-PixNav (1V)*는 ESCAPE-PixNav에서 active acquisition만 current-view로 대체하고 나머지 supervisor, memory와 async mechanism을 유지한다. 따라서 PixNav Native-1V와의 차이는 관측 수가 아니라 supervisor/grounding contract이고, ESCAPE-PixNav와의 차이는 task-conditioned active observation뿐이다. 같은 episode의 결과를 별도 Table V에 보고한다.

S2E reproduction. 공식 저장소가 제공하는 S2E-BC checkpoint를 동일 evaluator에서 먼저 재실행하여 protocol을 calibration한다. 이후 공개 BC를 초기화로 사용해 저자가 RL post-training한 *S2E-RL (ours)*를 평가한다. RAM 구조, trainable parameter, PPO objective, reward와 curriculum을 원 논문과 동일하게 재현한 범위는 본문의 실험 설정에 명시한다. 정확히 일치하지 않는 요소가 있으면 “independent reimplementaion” 대신 “our RL-post-trained S2E”로 표기한다. ESCAPE-S2E결합 시 S2E-RL weight는 frozen이다.

C. 구현 및 재현 설정

Method의 수식은 mechanism과 acceptance condition만 정의하고, 실제 실행에 사용한 모든 수치 parameter는 이 절에서 공개한다. 최종 평가 전 configuration을 고정하고 prompt/schema hash와 함께 보존한다. 현재 구현에서 확정된 값과 아직 실험 전 고정 필요 값은 Table II에 분리해 표시한다.

TABLE II
IMPLEMENTATION PARAMETERS. [TBD]는 최종 평가 전에 고정하여 공개한다.

항목	값
ObjectNav full-sweep heading offset	$\{0, \pm 60, \pm 120, -180\}^\circ$
PointNav initial directed sweep	goal align 후 $\{-30^\circ, 0^\circ, +30^\circ\}$
PointNav directed-view grid $(\bar{u}, \bar{v})$	$\bar{u} \in \{.25, .375, .5, .625, .75\}$, $\bar{v} \in \{.84, .90\}$
Panorama candidate grid $(\bar{u}, \bar{v})$	$\bar{u} \in \{.375, .5, .625\}$, $\bar{v} = .84$
ObjectNav grid/cap	directed $7 \times 2/14$, panorama $3 \times 2/4$
RGB screening output cap	view당 3 candidates, temperature 0
Pose interpolation bracket gap $\Delta r_{\max}^{\text{pose}}$	0.25 s
Pose parent/child frame	odom/base_link
VLM backbone set	Qwen3-VL-32B-Instruct-AWQ [20]; small-open/cross-family versions [TBD]
Strong VO baseline	exact public method/checkpoint and sensor input [TBD]
Place recognition	deterministic hash projection, 256-dimensional, cosine top-5, $\tau_{\text{pr}} = 0.86$, central processing unit (CPU)
Validated runtime	Ubuntu 24.04 x86-64; Python 3.9.23; PyTorch 2.8.0/CUDA 12.8; Habitat 0.3.3 headless
Graphics processing unit (GPU) host	2× NVIDIA RTX PRO 6000 Blackwell Server Edition (96 GiB each); driver 595.71.05
Decision TTL and displacement bounds	[TBD]
PixNav restoration/reobserve bounds	[TBD]
S2E handoff continuity bounds	[TBD]
Deadlock evidence	$\epsilon_s = .08$ m, $K_s = K_o = 3$, turn 120° , return 45° , history 20
Memory ranking weights	$\lambda_d = .55$, $\lambda_d = .90$, $c_{\max} = .5$, $\alpha_o = .25$, $\lambda_f = .1875$
Memory candidate cap	edge/frontier exits top-8, visual revisits top-5
Observation trigger	structured request 또는 blocked/no-progress/loop/empty candidate
Localization-noise severity ν	$\{0, .2, .5, 1\}$; 0은 oracle control
Corrupted localization state	planar position (p_x, p_y) and yaw ψ
Corruption-profile maxima	held-out Go2 calibration; stress bound [TBD]
Pose-stream delay levels	held-out Go2 median/95th percentile; stress value [TBD]

전체 framework와 Habitat runner는 Python 3.9.23으로 구현하며, VLM은 OpenAI-compatible application programming interface (API)로 독립 serving하여 simulator/navigation process와 분리한다. 현재 place-recognition module은 grayscale thumbnail, RGB statistics 및 histogram을 seeded random projection한 256-dimensional ℓ_2 -normalized descriptor를 사용한다. 이는 prototype retrieval descriptor이므로 top-5 revisit candidate만 제안하며, semantic/spatial 검증 없이 node를 hard merge하지 않는다. 입력 image resolution, camera intrinsic/extrinsic calibration, camera height와 pitch, observation novelty bin, controller action primitive, trajectory-context length와 sampling period, inference frequency, timeout, residual gain/clamp 및 backend별 acceptance threshold도 configuration table과 공개 run manifest에 기록한다. 실험 중 값을 변경하면 별도 configuration으로 취급하며 같은 표의 결과를 혼합하지 않는다. 현재 smoke configuration의 success radius는 1.0 m이지만, 최종 Habitat/Habitat-GS 평가는 동일한 저장 trajectory에 0.2 m와 1.0 m success criterion을 각각 적용하여 재계산한다. Smoke run의 기존 1.0 m summary를 Table III에 직접 혼합하지 않는다.

D. Mechanism ablations

본문 공간과 trial budget을 핵심 claim에 집중하기 위해 exhaustive 조합 대신 다음 최소 ablation을 사전 고정한다. Table VII은 동일 paired subset에서 full system의 구성요소 하나만 제거한 결과를 보고하고, latency sweep은 Fig. 4에 분리한다.

Active observation. Full task-conditioned policy와 forward current-view only를 비교한다. 이때 async transport와 memory는 동일하게 유지하여 active observation만 제거한다. Initial 360° only는 여유가 있을 때 수행하는 diagnostic이며 main claim의 필수 비교에는 포함하지 않는다. SR/SPL과 함께 candidate validity, progress rate, selection regret, views/query, body-yaw travel 및 view-acquisition interruption time을 보고한다.

Causal async와 representation-aware transport. 모든 variant는 같은 active-observation policy, directional memory, VLM contract와 frozen backend를 사용하고 scheduler/transport만 바꾼다. Sync는 capture부터 decision handoff까지 기존 navigation을 보류한다. Async-direct는 기존 trajectory를 계속 실행하지만 delayed pixel, metric goal과 rotation을 observation-time 표현 그대로 적용하며 representation transport와 causal admission을 사용하지 않는다. 이 intentionally unaligned variant는 simulator에서만 실행한다. Async+geometric rebase는 timestamped pose reconstruction과 metric/rotation rebase를 사용하지만 view-bound pixel의 restoration/re-observation 및 identity, freshness, owner, terminal fence는 제거한다. ESCAPE-Nav async는 Eqs. (8)–(12)의 representation별 transport, admission 및 atomic handoff를 모두 사용한다. Fig. 4는 latency 영향이 큰 Habitat-GS/S2E-RL에서 주입 latency에 따른 SPL과 normalized wall-clock curve를 표시한다. Table VII(a)는 두 environment/backend의 공통 $L = 2$ s endpoint에서 invalid/stale application, motion duty, completion time, query count와 application yield를 함께 보고하여 Sync의 stop-and-go cost와 Async-direct의 stale-application cost를 분리한다. Apply-time heading error는 별도 diagnostic으로 기록한다. AsyncShield의 pose transformation 일부만 구현한 경우 이를 AsyncShield reproduction이라 부르지 않고 Async+geometric rebase로 명명한다 [15].

Pose synchronization. Pose interpolation의 기여는 동일 pose stream에서 bracketing interpolation을 사용하는 full variant와 query 시점의 latest-available pose sample만 사용하는 variant를 비교한다. 별도 pose-stream delay는 held-out Go2 log의 median, 95th percentile 및 하나의 stress level로 고정하고, capture/apply transform error와 metric-goal error를 보고한다. 최종 지연값은 평가 전에 Table II에 공개한다.

VLM backbone portability. 기본 Qwen3-VL-32B-Instruct-AWQ, 더 작은 open multi-image VLM 및 다른 model family의 multi-image VLM을 비교한다. 비교 모델은 한 요청에서 순서가 보존된 여러 RGB image와 view identifier를 처리하고 공통 structured output을 반환할 수 있어야 하며, native video interface는 요구하지 않는다. 최종 평가 전에 small-open checkpoint와 cross-family model/API version을 고정한다. 모든 모델에는 동일한 ordered view bundle, heading, timestamp, task goal, candidate set, memory snapshot, prompt semantics, schema, decoding policy 및 paired episode를 제공하고 API

syntax 외의 model-specific prompt tuning은 허용하지 않는다. 먼저 recorded decision을 공통 $L = 2$ s에서 replay하여 semantic/contract quality를 비교하고, 이후 native latency로 실행하여 실제 inference time, stale rejection 및 application yield를 측정한다. 각 backbone에서 ESCAPE-PixNav (1V)와 ESCAPE-PixNav를 짝지어 비교한다.

Memory. Controlled recovery subset에서 no memory, undirected action–outcome memory 및 full directional graph만 비교한다. Detector recall, false triggers/episode, DRS, FBR와 oscillation count를 함께 보고하여 검출, 탈출과 재진입 억제를 분리한다. 모든 ratio에는 검출 deadlock, annotated deadlock 및 revisit opportunity의 raw count를 병기한다. Flat history와 failed-node blacklist는 trial budget에 여유가 있을 때만 추가 diagnostic으로 실행한다. Place association은 annotated revisit에서 retrieval recall@5, semantic/spatial verification이 승인한 same-place relation의 precision 및 false-association count를 보고한다. Hard merge 대 duplicate+soft-link는 이 event-level 검증과 분리된 diagnostic으로 둔다.

E. Matched localization degradation

DD-PPO에만 noise를 주고 ESCAPE-Nav에는 clean PointGoal 또는 ground-truth apply pose를 남기면 불공정하다. 따라서 모든 방법의 goal, odometry 및 rebase pose를 같은 corrupted state estimate로 계산한다. $\nu = 0$ 은 실제 운용 조건이 아니라 simulator와 metric implementation을 확인하는 oracle control로 명시하고, calibration log가 정의하는 최소 nonzero operating condition을 $\nu = 0.2$ 로 둔다. Table III은 oracle clean과 $\nu \in \{0.2, 0.5, 1\}$ 의 exact SR/SPL을 모두 공개하여 clean 성능을 숨기지 않으면서 noise-free GPS가 실제 조건이라는 인상도 피한다.

$$\tilde{\mathbf{x}}_t = \mathbf{x}_t + \mathbf{b}_t + \boldsymbol{\epsilon}_t, \quad (17)$$

$$\mathbf{b}_t = \mathbf{b}_{t-1} + \boldsymbol{\eta}_t. \quad (18)$$

여기서 $\mathbf{x}_t = [p_x, p_y, \psi]^\top$ 와 $\tilde{\mathbf{x}}_t$ 는 각각 clean planar localization pose와 모든 방법에 공통으로 제공하는 corrupted pose estimate이며, $\mathbf{b}_t$ 는 position/yaw 누적 bias다. $\boldsymbol{\epsilon}_t$ 와 $\boldsymbol{\eta}_t$ 는 각각 zero-mean white-noise sample과 random-walk increment이며, 분포와 scale은 noise severity별로 고정한다. Corruption profile은 planar position 및 yaw의 white noise와 random-walk drift, pose-update dropout에 따른 hold-last estimate, sparse position/yaw jump를 조합한다. 이 pose로 DD-PPO의 PointGoal/GPS–Compass state, 다른 방법의 odometry-relative goal 및 async rebase pose를 모두 계산하고, episode별 동일 trace를 재사용한다. 따라서 Table III의 noise는 RGB corruption, actuation noise, VLM inference latency 또는 GPS-message latency가 아니라 matched localization-pose corruption이다. Success radius $r \in \{0.2, 1.0\}$ m와 noise severity $\nu \in \{0, .2, .5, 1\}$ 에 대해

$$\text{SPL}_r(\nu) = \frac{1}{N} \sum_{i=1}^N S_i^{(r)}(\nu) \frac{\ell_i}{\max(p_i(\nu), \ell_i)}. \quad (19)$$

여기서 $S_i^{(r)}(\nu)$ 는 episode i 가 반경 r 에서 성공하면 1, 아니면 0인 indicator이고, ℓ_i 와 $p_i(\nu)$ 는 각각 shortest-path length와 실제 path length다. 따라서 실패 episode도 평균의 분모 N 에

포함되며 SPL 기여는 0이다. Table III은 Habitat 및 Habitat-GS에서 네 severity의 SR₂, SR_{1.0}, SPL₂ 및 SPL_{1.0}을 직접 보고한다.

Noise profile은 final test episode와 분리된 calibration trajectory에서 고정한다. Reference pose와 Go2 odometry의 per-meter translation/yaw drift, update dropout duration, sparse jump magnitude와 event rate를 측정하고 각 최대값을 calibration distribution의 95th percentile로 정한다. White-noise 및 random-walk 표준편차, jump magnitude, dropout probability와 jump rate는 ν 에 따라 clean에서 이 최대 profile까지 선형 보간한다. 물리 calibration 범위를 넘는 추가 실험은 *stress*로 명시하고 realistic noise와 구분한다. Physical log가 충분하지 않으면 별도 pilot subset의 frozen direct-goal baseline에서 severe SPL retention이 floor에 도달하지 않는 범위를 선택하되, ESCAPE-Nav결과나 final test episode를 severity 선정에 사용하지 않는다. 각 nonzero severity에서 episode당 최소 세 corruption seed를 생성하고, 같은 episode-seed trace를 모든 방법에 재사용한다. Profile, seed와 episode 별 trace는 configuration manifest에 공개한다.

각 corruption factor의 full severity sweep을 모든 방법에 반복하지 않는다. 원인 분석은 $\nu = 0.5$ 에서 position-only, yaw-only 및 dropout/jump-only trace를 DD-PPO, 동일 backend의 direct-goal variant와 full ESCAPE-Nav에만 적용한다. Main robustness claim은 calibrated composite severity에 따른 SR/SPL 변화로 판단하고 factor-isolation 결과는 failure analysis로 제한한다. GPS latency는 GPS를 사용하지 않는 방법과 직접 대응하지 않으므로 별도 sensor-specific benchmark로 만들지 않는다. 대신 모든 방법에 공통인 pose-stream delay를 위 pose-synchronization ablation에서 평가한다. RGB low contrast, glare, blur 및 repeated texture stress는 localization 결과와 혼합하지 않고 별도 qualitative stress로 보고한다.

F. Latency protocol, 지표 및 통계

VLM delay는 $L \in \{0, 0.5, 1, 2, 4, 6\}$ s로 주입한다. Primary sweep은 paired closed-loop rollout이다. 각 run에서 실제 생성된 query를 temperature zero로 한 번 decoding하여 response payload와 observation identity를 고정하고, payload를 바꾸지 않은 채 delivery timestamp에 L 만 더한다. Scheduler별 trajectory가 달라지면 이후 query도 달라질 수 있으므로 서로 다른 scheduler에 하나의 고정 observation trace를 강제로 재사용하지 않는다. 대신 동일 episode, initial state, simulator seed, model, prompt/schema와 decoding policy를 공유한다. 별도의 fixed-trace microbenchmark는 동일 recorded response의 stale rejection, pose error와 handoff event만 검증하며 navigation SPL 근거로 사용하지 않는다. Fig. 4는 closed-loop latency sensitivity를 보고한다. 이후 model/version과 server를 고정하고 strict schema-constrained decoding을 사용하는 live VLM subset으로 native end-to-end latency, query count, input images/episode, application yield와 useful applications/min을 검증한다. Prompt 및 schema version/hash를 각 run에 기록하며 mechanism ablation에서는 model과 prompt를 모두 고정한다. VLM-backbone ablation은 공통 prompt/schema를 유지하되 backbone만 교체하고 fixed-latency와 native-latency 결과를 분리한다. Burst delay, response drop과 out-of-order completion은 primary curve와 분리한 stress case로 보고한다.

Latency가 path quality뿐 아니라 completion efficiency에 미치는 영향을 실패 episode까지 포함해 비교한다. Strict 0.2 m success indicator를 S_i 라 하고 공통 wall-clock timeout을 $T_{\max}$ 라 할 때, episode i 의 집계 시간과 method m 의 normalized completion time은

$$T_i^\dagger = S_i \min(T_i, T_{\max}) + (1 - S_i)T_{\max},$$

$$\bar{T}_m(L) = \frac{1}{N} \sum_{i=1}^N \frac{T_{i,m}^\dagger(L)}{T_{\max}}. \quad (20)$$

으로 정의한다. 따라서 실패, collision termination 및 timeout은 모두 1의 normalized time cost를 가지며 조기 실패가 빠른 완료로 오인되지 않는다. Fig. 4(b)는 $\bar{T}_m(L)$ 와 paired bootstrap 95% CI를, Table VII(a)는 $L = 2$ s의 실제 second 단위 $T_i^\dagger$ 평균을 보고한다.

표준 metric은 success rate (SR), success weighted by path length (SPL), final distance와 collision/m이다 [8]. PointNav의 SR과 SPL은 strict 0.2 m와 near-goal 1.0 m success radius에서 모두 계산하며, Eq. (19)처럼 실패 episode도 SPL 평균에 포함한다. 0.2 m 결과를 primary criterion으로 해석하고 1.0 m 결과는 near-goal 도달과 terminal precision을 구분하는 보조 기준으로 사용한다. Async metric은 observation-to-application latency, stale rejection, applied/completed decision ratio, Eq. (20)의 wall-clock completion time, translation duty cycle, post-capture inference-execution overlap ratio, queries/episode와 executor/controller frequency이다. Active-observation metric의 views/query는 supervisor query 하나에 실제 제공된 RGB frame 수의 평균이고, yaw/query는 해당 query를 위해 실행한 body rotation의 절댓값 합을 degree로 나타낸 평균이다. 이와 함께 view-acquisition interruption time, candidate validity, progress rate, selection regret와 invalid view application을 기록한다. Memory/recovery metric은 detector recall (DetR), false-trigger rate (FTR), directional recovery score (DRS)와 failed-branch re-entry rate (FBR)로 정의한다.

$$\text{DetR} = \frac{N_{\text{true detected}}}{N_{\text{annotated deadlock}}}, \quad \text{FTR} = \frac{N_{\text{false trigger}}}{N_{\text{episode}}},$$

$$\text{DRS} = \frac{N_{\text{escaped and resumed}}}{N_{\text{true detected}}}, \quad \text{FBR} = \frac{N_{\text{failed edge reentry}}}{N_{\text{opportunity}}}. \quad (21)$$

$N_{\text{annotated deadlock}}$ 은 controlled suite가 정의한 실제 deadlock 수, $N_{\text{true detected}}$ 는 그중 detector가 검출한 수다. 각 annotated interval에는 detector의 첫 false-to-true transition을 최대 하나만 대응시키고, 어떤 interval과도 겹치지 않는 transition을 false trigger로 센다. 평가 logger는 reset 이전의 episode/decision identifier, timestamp, confirmed flag와 reason set을 저장하며, 이 필드가 누락된 run은 DetR/FTR 집계에서 제외한다. $N_{\text{escaped and resumed}}$ 는 true detection 후 탈출하여 original-goal pursuit를 재개한 수다. $N_{\text{opportunity}}$ 는 실패 edge를 다시 선택할 수 있었던 annotated revisit event 수이고, $N_{\text{failed edge reentry}}$ 는 그중 실제로 같은 실패 edge에 재진입한 수다. 따라서 DetR이 낮은 variant가 쉬운 event만으로 높은 DRS를 얻는 것을 구분할 수 있다. Place association은 annotated same-place query의 retrieval recall@5, 최종 승인 relation의 precision과 false-association count를 별도로 보고한다. 모든 비교는 paired episode로 수행한다. SR에는 Wilson interval과 McNemar test

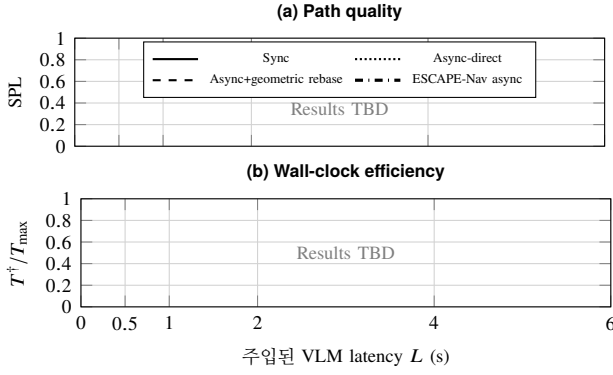


Fig. 4. VLM latency에 따른 path quality와 wall-clock efficiency. Habitat-GS/S2E-RL의 paired closed-loop run별 deterministic response는 바꾸지 않고 delivery delay L 만 주입한다. (a)는 SPL, (b)는 Eq. (20)의 timeout-normalized time이며 95% CI를 표시한다. Table VII은 $L = 2$ s endpoint와 query/application cost를 보고한다.

를, SPL/시간에는 scene-episode-corruption-seed 계층을 보존하는 hierarchical paired bootstrap 95% confidence interval (CI)을 사용한다. Clean track은 scene-episode 계층을 사용하며, recovery ratio에는 event denominator와 bootstrap CI를 함께 보고한다. 결과 확인 전 primary endpoint를 RQ1의 strict-radius SPL 및 progress rate, RQ2의 strict-radius SPL, normalized completion time 및 invalid/stale application, RQ3의 DRS와 FBR로 고정한다. 각 RQ 내부의 사전 지정 pairwise comparison에는 Holm correction을 적용하고, 나머지 metric과 stress condition은 effect size와 CI를 제시하는 secondary analysis로 명시한다. 실패 run, excluded simulator error, completed/applied/rejected query와 recovery-event raw denominator를 method별로 모두 공개한다.

V. 결과 분석 계획과 논의

A. 사전 가설과 반증 조건

H1: task-conditioned observation은 ESCAPE-Nav (1V)보다 progress/SPL을 높이고 regret를 낮춰야 하며, 이득 없이 view/yaw cost만 늘면 claim을 축소한다. **H2:** latency 증가 시 Sync는 Eq. (20)의 time, Async-direct는 stale/SPL이 악화되고 ESCAPE-Nav async는 둘을 억제해야 한다. Geometric rebase와 차이가 없으면 restoration/fence 기여를 축소한다. **H3:** directional graph는 no/undirected memory보다 DRS를 높이고 FBR/oscillation을 낮추되 Eq. (21)의 DetR/FTR를 유지해야 한다. Retrieval/approval 정확도가 낮으면 oracle association 조건으로 제한한다. **H4:** HM3D에서는 PixNav, Habitat-GS에서는 S2E-RL이 유리할 것으로 가정되 각 ESCAPE-Nav row는 같은 backend의 Direct-goal과 비교한다. Frozen indoor DD-PPO/VO는 outdoor 참고값이며, NavBench-GS rerun이 calibration되지 않으면 reported S2E-Full과 직접 비교하지 않는다. **H5:** 세 backbone에서 schema/grounding validity와 ESCAPE-Nav (1V) 대비 개선이 유지되지 않으면 plug-and-play 범위를 축소한다. **H6:** Go2는 paired core scenario만 비교 우위의 근거로 사용한다.

Place association은 Table VII(c)와 별도로 annotated query 수, retrieval recall@5, same-place approval precision과 false-

association count를 보고하여 retrieval과 verification failure를 분리한다.

B. VLM backbone portability 해석

Table VI의 fixed-latency replay는 HM3D와 Habitat-GS에서 reasoning/grounding 차이를, native-latency 실행은 실제 배치 비용을 평가한다. 따라서 더 느린 backbone의 native SR 저하를 semantic failure로 해석하지 않으며 schema-valid response, provided candidate를 선택한 비율, stale rejection 및 application yield를 함께 본다. 세 backbone 모두에서 동일 prompt 의미와 output schema를 유지하고 API serialization만 달리한다. Full ESCAPE-Nav의 효과는 backbone 간 절대 점수가 아니라 각 backbone과 environment 내부의 paired ESCAPE-Nav (1V) 대비 차이와 confidence interval로 판단한다. 두 environment를 합친 latency/yield macro-average는 environment별 navigation score를 대체하지 않는다.

C. S2E reproduction과 통계적 해석

Table IV의 †와 ‡는 reported/rerun source이며 SR/RC/CT는 네 condition의 macro-average다. RAM, trainable block, PPO, reward, curriculum과 train distribution이 일치할 때만 “independent reimplement”으로 기술하고, 아니면 “our RL-post-trained S2E”로 제한한다. 이 결과는 Table III의 same-backend 비교와 분리한다. HM3D는 고정 100–200 paired episode와 CI를 보고하고, memory claim은 recovery suite의 annotated event와 Eq. (21)로 검증한다.

D. 정성 분석과 실로봇 배치

Fig. 6는 Table III의 수치와 같은 method set에 대해 HM3D와 Habitat-GS의 preselected paired episode를 top-view trajectory로 비교한다. 이는 overall path behavior를 보여주는 보조 자료이며 두 사례로 일반 성능을 주장하지 않는다. Deadlock과 oscillation은 단일 RGB frame으로 식별하기 어려운 temporal event이므로 정량 근거는 Table VII(c)의 DetR/FTR/DRS/FBR/Osc.와 raw event count로 제한한다. 필요한 failure case는 timestamp가 표시된 top-view trajectory와 event log로 설명하되 정량 결과를 대체하지 않는다.

Fig. 5은 Unitree Go2, 고정 전방 RGB camera와 평가 환경이 함께 보이는 대표 deployment 사진 한 장으로 간결하게 제시한다. 세 core scenario의 paired 5회 결과는 Table VIII에 보고하고, active-view recovery와 dynamic-obstacle의 Full-only 추가 trial은 paired 우월성 근거가 아닌 deployment 범위 확인으로 본문에 분리한다.

E. 한계

ESCAPE-Nav는 globally consistent localization이나 formal collision safety를 보장하지 않는다. 큰 pose drift는 passive rebase와 graph association을 함께 손상시킨다. 특히 RGB-only observation은 직접적인 거리와 surface geometry를 제공하지 않으므로 얇거나 낮은 장애물, texture가 불명확한 경계 및 non-planar terrain에서 traversability 판단과 collision avoidance가 취약할 수 있다. Body-yaw observation은 회전 비용과 locomotion disturbance를 유발하고, ground-plane projection은 calibration과 locally planar terrain에 의존한다. 이는 optional depth를 candidate grounding, obstacle validation 및 transport

TABLE III

HABITAT POINTNAV MAIN-RESULT TEMPLATE. $\nu = 0$ 은 ORACLE CONTROL이며 실제 운용의 NOISE-FREE GPS를 가정하지 않는다. 각 SEVERITY에서 SR과 SPL은 0.2 m 및 1.0 m SUCCESS RADIUS로 계산한다. HABITAT-GS의 INDOOR-TRAINED FROZEN POLICY는 참고값이며, PRIMARY COMPARISON은 동일 BACKEND의 DIRECT-GOAL 대 ESCAPE-NAV다.

Method	Oracle ($\nu = 0$)				Low ($\nu = .2$)				Moderate ($\nu = .5$)				Severe ($\nu = 1$)			
	SR _{.2}	SR _{1.0}	SPL _{.2}	SPL _{1.0}	SR _{.2}	SR _{1.0}	SPL _{.2}	SPL _{1.0}	SR _{.2}	SR _{1.0}	SPL _{.2}	SPL _{1.0}	SR _{.2}	SR _{1.0}	SPL _{.2}	SPL _{1.0}
<i>HM3D</i>																
DD-PPO RGB	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Strong VO PointNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Direct-goal PixNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-PixNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Direct-goal S2E-RL	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-S2E	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
<i>Habitat-GS</i>																
DD-PPO RGB	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Strong VO PointNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Direct-goal PixNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-PixNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Direct-goal S2E-RL	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-S2E	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

TABLE IV

NAV BENCH-GS REPRODUCTION. SR/RC/CT의 FOUR-CONDITION MACRO-AVERAGE이며 RC와 CT는 S2E PROTOCOL을 따른다. †: REPORTED; ‡: RERUN.

Method	SR ↑	RC ↑	CT ↓
S2E-BC [†]	0.59	0.76	1.23
S2E-Full [†]	0.68	0.83	0.84
S2E-BC rerun [‡]	[TBD]	[TBD]	[TBD]
Direct S2E-RL [‡]	[TBD]	[TBD]	[TBD]
ESCAPE-S2E [‡]	[TBD]	[TBD]	[TBD]

TABLE V

HM3D OBJECTNAV TRANSFER. 6V는 단일 전방 CAMERA의 FULL-ROTATION SIX-VIEW SWEEP 이고, 1V는 전방 한 장이다.

Method	SR ↑	SPL ↑
PixNav Native-6V	[TBD]	[TBD]
PixNav Native-1V	[TBD]	[TBD]
ESCAPE-PixNav (1V)	[TBD]	[TBD]
ESCAPE-PixNav	[TBD]	[TBD]
ESCAPE-S2E	[TBD]	[TBD]

Actual Unitree Go2 deployment image

TBD: robot, forward RGB camera and evaluation scene

Fig. 5. Unitree Go2의 대표 deployment 장면. 최종본에서는 로봇, 고정 전방 RGB camera와 paired core-scenario 환경이 함께 보이는 실제 사진 한 장으로 교체한다. 정량 결과는 Table VIII에 보고한다.

uncertainty에 결합하는 depth-aware extension의 직접적인 동기다. Raw pixel은 큰 translation 또는 scene change 후 재사용할 수 없으며, sparse graph에는 global pose-graph optimization이 없다. 계속 갱신되는 relative PointGoal을 사용하는 Habitat 설정은 완전한 GNSS-denied navigation보다 쉽고, ObjectNav 및 polar-inspired corruption은 실제 극지 성능을 대체하지 않는다.

VI. 결론

본 논문은 단일 전방 카메라와 느린 VLM을 사용하는 navigation을 temporal, representational, experiential causal

consistency 문제로 정식화하고 ESCAPE-Nav를 제안하였다. ObjectNav의 초기 panorama, PointNav의 goal-aligned directed sweep 및 recovery-triggered full sweep은 필요한 시야를 선택적으로 확보한다. World goal은 passive transform, selected pixel은 active viewpoint restoration 또는 재 관측, metric subgoal은 apply-time rebase를 거치며, identity, freshness, ownership과 terminal lifecycle을 통과한 decision만 fast executor에 전달된다. Acknowledged directed transition에 failure와 escape를 독립적으로 기록함으로써 실패한 branch 재진입을 억제하도록 설계하고, 같은 contract를 서로 다른 VLM backbone, PixNav, independently RL-post-trained S2E 및 Unitree Go2에 추가 joint tuning 없이 연결한다. 최종 실험은 Habitat/HM3D indoor와 Habitat-GS outdoor의 paired PointNav 비교, 독립적인 NavBench-GS 재현, VLM-backbone ablation, recovery suite와 실로봇 log에서 각 mechanism의 효과와 비용을 분리해 검증한다. 향후에는 optional depth를 candidate grounding, traversability 검증 및 metric transport uncertainty에 결합하는 depth-aware extension으로 RGB-only framework의 geometric robustness를 확장할 것이다.

REFERENCES

- [1] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, “DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames,” in *International Conference on Learning Representations*, 2020.
- [2] X. Zhao, H. Agrawal, D. Batra, and A. G. Schwing, “The surprising effectiveness of visual odometry techniques for embodied pointgoal navigation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 16 127–16 136.
- [3] R. Partsey, E. Wijmans, N. Yokoyama, O. Dobosevych, D. Batra, and O. Maksymets, “Is mapping necessary for realistic pointgoal navigation?” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 17 232–17 241.
- [4] R. Bajcsy, “Active perception,” *Proceedings of the IEEE*, vol. 76, no. 8, pp. 996–1005, 1988.
- [5] S. Chen, Y. Li, and N. M. Kwok, “Active vision in robotic systems: A survey of recent developments,” *The International Journal of Robotics Research*, vol. 30, no. 11, pp. 1343–1377, 2011.
- [6] X. Zhao, W. Cai, L. Tang, and T. Wang, “ImagineNav: Prompting vision-language models as embodied navigator through scene imagination,” *arXiv preprint arXiv:2410.09874*, 2024.

TABLE VI

CROSS-DOMAIN VLM-BACKBONE PORTABILITY TEMPLATE. HM3D와 HABITAT-GS의 SR/SPL 및 OUTPUT VALIDITY는 ENVIRONMENT별 고정 PAIRED SUBSET의 공통 $L = 2$ s REPLAY에서 측정하고, LATENCY와 APPLICATION YIELD는 두 SUBSET의 EQUAL-WEIGHT NATIVE-LATENCY MACRO-AVERAGE다. 모든 BACKBONE은 동일 PROMPT/SCHEMA와 ORDERED MULTI-VIEW RGB BUNDLE를 사용하며 정확한 CHECKPOINT/API VERSION은 최종 평가 전에 고정한다.

VLM backbone	Supervisor	H-SR $\uparrow$	H-SPL $\uparrow$	G-SR $\uparrow$	G-SPL $\uparrow$	Schema valid $\uparrow$	Ground valid $\uparrow$	Latency $\downarrow$	Apply yield $\uparrow$
Qwen3-VL-32B-Instruct-AWQ	ESCAPE-Nav (1V)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
	Full ESCAPE-Nav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Small open multi-image VLM	ESCAPE-Nav (1V)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
	Full ESCAPE-Nav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Cross-family multi-image VLM	ESCAPE-Nav (1V)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
	Full ESCAPE-Nav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

TABLE VII

세 핵심 MECHANISM의 분리 ABLATION TEMPLATE. H와 G는 각각 HM3D/ESCAPE-PIXNAV와 HABITAT-GS/ESCAPE-S2E의 PAIRED SUBSET이다. 따라서 (b)의 ESCAPE-Nav (1V)는 H에서 ESCAPE-PIXNAV (1V), G에서 ESCAPE-S2E (1V)를 뜻한다. (a)는 공통 $L = 2$ s에서 CAUSAL EXECUTION과 POSE SYNCHRONIZATION, (b)는 COARSE POINTGOAL의 EXECUTABLE-CANDIDATE REFINEMENT와 ACTIVE-OBSERVATION COST, (c)는 ANNOTATED RECOVERY SUITE의 DIRECTIONAL MEMORY를 평가한다. 모든 RATIO에는 RAW DENOMINATOR와 95% CI를 병기하며 FULL LATENCY CURVE는 FIG. 4에 보고한다.

(a) Causal execution and pose synchronization							
구성	H-SPL $\uparrow$	G-SPL $\uparrow$	Inv./stale $\downarrow$	Duty $\uparrow$	Time (s) $\downarrow$	Q/ep $\downarrow$	Yield $\uparrow$
Sync	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Async-direct	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Async+geometric rebase	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-Nav async	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Latest-pose lookup (no interpolation)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

(b) Active observation and candidate grounding							(c) Directional memory and recovery					
구성	H-Prog. $\uparrow$	G-Prog. $\uparrow$	Valid $\uparrow$	Regret $\downarrow$	Views/q $\downarrow$	Yaw/q $\downarrow$	구성	DetR $\uparrow$	FTR $\downarrow$	DRS $\uparrow$	FBR $\downarrow$	Osc. $\downarrow$
Direct-goal	[TBD]	[TBD]	[TBD]	[TBD]	–	–	No memory	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-Nav (1V)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	Undirected memory	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Task-conditioned active	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	Directional memory	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

TABLE VIII

UNITREE Go2 PAIRED CORE-SCENARIO TEMPLATE. 동일 ROUTE/START/GOAL/TIMEOUT에서 각 METHOD를 5회 실행한다. SUCC.와 IF는 SUCCESS/INTERVENTION-FREE TRIAL 수, TIME은 FAILURE를 TIMEOUT으로 집계한 $T^\dagger$, REC. SUCC.는 SUCCESSFUL/TRIGGERED RECOVERY EVENT, RE-ENTRY는 FAILED-EDGE 재진입 수다.

Scenario	Method	Succ./5 $\uparrow$	IF/5 $\uparrow$	Time (s) $\downarrow$	Duty $\uparrow$	Rec. succ. $\uparrow$	Re-entry $\downarrow$
Dead-end room	Direct-goal	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
	Full ESCAPE-Nav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Blocked goal direction	Direct-goal	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
	Full ESCAPE-Nav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Repeated corridor	Direct-goal	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
	Full ESCAPE-Nav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

- [7] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, "Habitat: A platform for embodied ai research," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9339–9347.
- [8] P. Anderson, A. Chang, D. S. Chaplot, A. Dosovitskiy, S. Gupta, V. Koltun, J. Kosecka, J. Malik, R. Mottaghi, M. Savva, and A. Zamir, "On evaluation of embodied navigation agents," *arXiv preprint arXiv:1807.06757*, 2018.
- [9] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. Turner, E. Undersander, W. Galuba, A. Westbury, A. Chang, M. Savva, Y. Zhao, and D. Batra, "Habitat-matterport 3d dataset (HM3D): 1000 large-scale 3d environments for embodied ai," in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [10] W. Cai, S. Huang, G. Cheng, Y. Long, P. Gao, C. Sun, and H. Dong, "Bridging zero-shot object navigation and foundation models through pixel-guided navigation skill," *arXiv preprint arXiv:2309.10309*, 2023.
- [11] H. He, Y. Ma, B. Squicciarini, W. Wu, and B. Zhou, "From seeing to experiencing: Scaling navigation foundation models with reinforcement learning," in *International Conference on Learning Representations*, 2026, arXiv:2507.22028v2.
- [12] Z. Xia, J. Xu, C. Cui, Y. Yu, J. Zhang, Q. Yan, T. Ni, J. Chen, X. Zhou, H. Bao, R. Hu, and S. Peng, "Habitat-GS: A high-fidelity navigation simulator with dynamic gaussian splatting," *arXiv preprint arXiv:2604.12626*, 2026.
- [13] N. Hirose, C. Glossop, D. Shah, and S. Levine, "AsyncVLA: An asynchronous vla for fast and robust navigation on the edge," *arXiv preprint arXiv:2602.13476*, 2026.
- [14] C. Xiong, L. Wei, X. Hu, K. Ma, Z. Xia, Z. Jiang, Z. Sun, and L. Pei, "SFCo-Nav: Efficient zero-shot visual language navigation via collaboration of slow llm and fast attributed graph alignment," *arXiv preprint arXiv:2603.01477*, 2026.
- [15] K. Yang, Z. Chu, Y. Guo, Z. Wang, S. Xie, Y. Shen, X. Wu, X. Li, and M. Xu, "AsyncShield: A plug-and-play edge adapter for asynchronous cloud-based VLA navigation," *arXiv preprint arXiv:2604.24086*, 2026.
- [16] F. Wu, W. Zuo, W. Yang, J. Xiao, Y. Liu, and X. Zeng, "ReMemNav: A rethinking and memory-augmented framework for zero-shot object navigation," *arXiv preprint arXiv:2603.26788*, 2026.
- [17] Z. Ge, X. Jia, C. Wu, Y. Zhou, Z. Wu, and Y.-G. Jiang, "EvoMemNav: Efficient self-evolving fine-grained memory for zero-shot embodied navigation," *arXiv preprint arXiv:2606.03509*, 2026.
- [18] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: An open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.

	DD-PPO RGB	Direct-goal PixNav	ESCAPE-PixNav	Direct-goal S2E-RL	ESCAPE-S2E
<i>HM3D</i>	Top-view trajectory TBD	Top-view trajectory TBD	Top-view trajectory TBD	Top-view trajectory TBD	Top-view trajectory TBD
<i>Habitat-GS</i>	Top-view trajectory TBD	Top-view trajectory TBD	Top-view trajectory TBD	Top-view trajectory TBD	Top-view trajectory TBD

Fig. 6. Table III의 방법별 top-view trajectory 비교. 각 row는 결과 확인 전에 고정된 동일 environment의 paired episode, start pose와 goal을 사용한다. 모든 panel은 동일 map extent와 scale로 start, goal, executed trajectory, terminal position 및 collision event를 표시한다. 위는 HM3D, 아래는 Habitat-GS이며 왼쪽부터 DD-PPO RGB, Direct-goal PixNav, ESCAPE-PixNav, Direct-goal S2E-RL 및 ESCAPE-S2E다.

- [19] D. Batra, A. Gokaslan, A. Kembhavi, O. Maksymets, R. Mottaghi, M. Savva, A. Toshev, and E. Wijmans, “Objectnav revisited: On evaluation of embodied agents navigating to objects,” *arXiv preprint arXiv:2006.13171*, 2020.
- [20] S. Bai *et al.*, “Qwen3-VL technical report,” *arXiv preprint arXiv:2511.21631*, 2025.